

SQL QUERY OPERATIONS

Project: Doctor Appointment Booking System

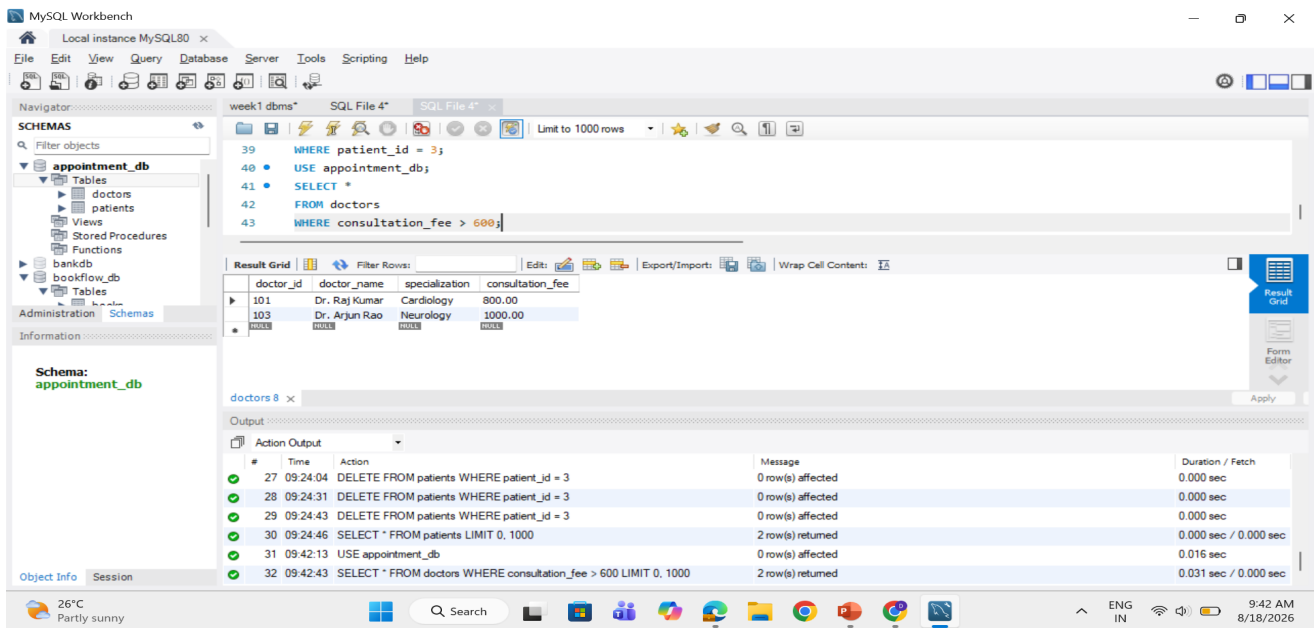
Database: appointment_db

Aim: To perform filtering, grouping, sorting, set operations, and aggregate functions using SQL.

1. WHERE Clause

The WHERE clause filters records according to a specified condition.

```
SELECT * FROM doctors
WHERE consultation_fee > 600;
```



The screenshot displays the MySQL Workbench interface. The SQL Editor window contains the following query:

```
39 WHERE patient_id = 3;
40 USE appointment_db;
41 SELECT *
42 FROM doctors
43 WHERE consultation_fee > 600;
```

The Results window shows the output of the query, displaying a table with the following data:

doctor_id	doctor_name	specialization	consultation_fee
101	Dr. Raj Kumar	Cardiology	800.00
103	Dr. Arjun Rao	Neurology	1000.00

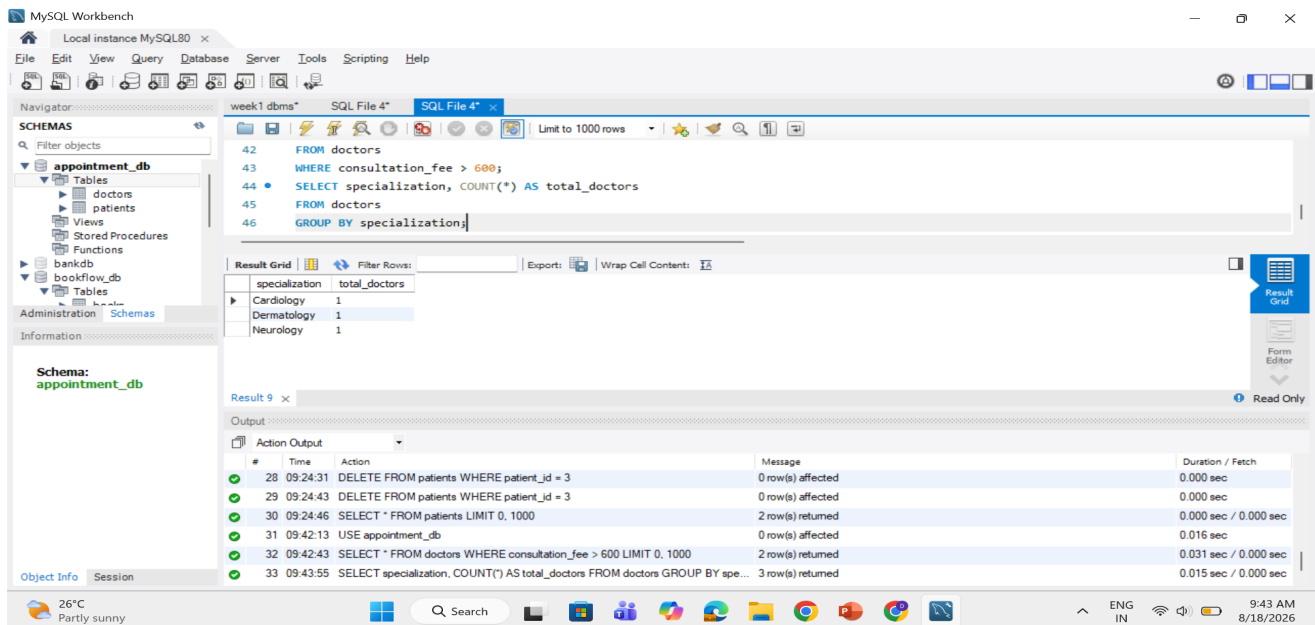
The Output window shows the execution log, including the following entries:

#	Time	Action	Message	Duration / Fetch
27	09:24:04	DELETE FROM patients WHERE patient_id = 3	0 row(s) affected	0.000 sec
28	09:24:31	DELETE FROM patients WHERE patient_id = 3	0 row(s) affected	0.000 sec
29	09:24:43	DELETE FROM patients WHERE patient_id = 3	0 row(s) affected	0.000 sec
30	09:24:46	SELECT * FROM patients LIMIT 0, 1000	2 row(s) returned	0.000 sec / 0.000 sec
31	09:42:13	USE appointment_db	0 row(s) affected	0.016 sec
32	09:42:43	SELECT * FROM doctors WHERE consultation_fee > 600 LIMIT 0, 1000	2 row(s) returned	0.031 sec / 0.000 sec

2. GROUP BY Clause

GROUP BY groups rows having the same value and can be used with aggregate functions.

```
SELECT specialization, COUNT(*) AS total_doctors
FROM doctors
GROUP BY specialization;
```



The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
42 FROM doctors
43 WHERE consultation_fee > 600;
44 SELECT specialization, COUNT(*) AS total_doctors
45 FROM doctors
46 GROUP BY specialization;
```

The Results window displays the following data:

specialization	total_doctors
Cardiology	1
Dermatology	1
Neurology	1

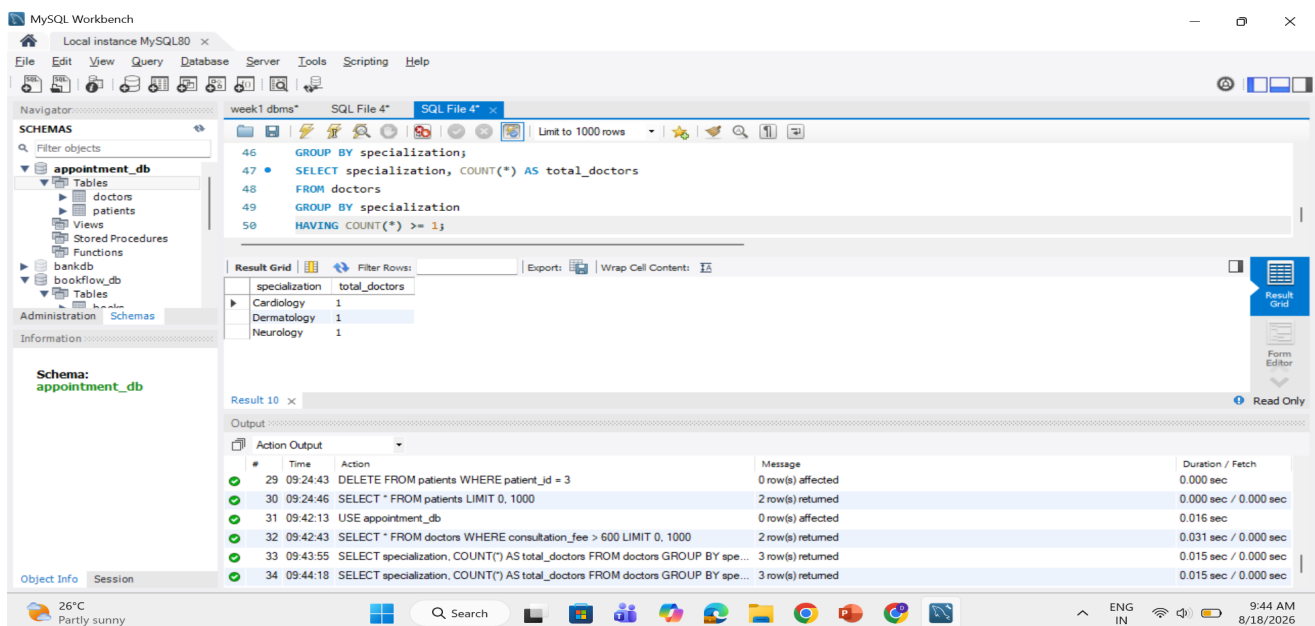
The Output window shows the execution log with the following entries:

#	Time	Action	Message	Duration / Fetch
28	09:24:31	DELETE FROM patients WHERE patient_id = 3	0 row(s) affected	0.000 sec
29	09:24:43	DELETE FROM patients WHERE patient_id = 3	0 row(s) affected	0.000 sec
30	09:24:46	SELECT * FROM patients LIMIT 0, 1000	2 row(s) returned	0.000 sec / 0.000 sec
31	09:42:13	USE appointment_db	0 row(s) affected	0.016 sec
32	09:42:43	SELECT * FROM doctors WHERE consultation_fee > 600 LIMIT 0, 1000	2 row(s) returned	0.031 sec / 0.000 sec
33	09:43:55	SELECT specialization, COUNT(*) AS total_doctors FROM doctors GROUP BY spe...	3 row(s) returned	0.015 sec / 0.000 sec

3. HAVING Clause

HAVING filters the groups produced by GROUP BY.

```
SELECT specialization, COUNT(*) AS total_doctors
FROM doctors
GROUP BY specialization
HAVING COUNT(*) >= 1;
```



The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
46 GROUP BY specialization;
47 SELECT specialization, COUNT(*) AS total_doctors
48 FROM doctors
49 GROUP BY specialization
50 HAVING COUNT(*) >= 1;
```

The Results window displays the following data:

specialization	total_doctors
Cardiology	1
Dermatology	1
Neurology	1

The Output window shows the execution log with the following entries:

#	Time	Action	Message	Duration / Fetch
29	09:24:43	DELETE FROM patients WHERE patient_id = 3	0 row(s) affected	0.000 sec
30	09:24:46	SELECT * FROM patients LIMIT 0, 1000	2 row(s) returned	0.000 sec / 0.000 sec
31	09:42:13	USE appointment_db	0 row(s) affected	0.016 sec
32	09:42:43	SELECT * FROM doctors WHERE consultation_fee > 600 LIMIT 0, 1000	2 row(s) returned	0.031 sec / 0.000 sec
33	09:43:55	SELECT specialization, COUNT(*) AS total_doctors FROM doctors GROUP BY spe...	3 row(s) returned	0.015 sec / 0.000 sec
34	09:44:18	SELECT specialization, COUNT(*) AS total_doctors FROM doctors GROUP BY spe...	3 row(s) returned	0.015 sec / 0.000 sec

4. Aggregate Functions

MIN, MAX, COUNT, SUM and AVG summarize values in a table.

```
SELECT MIN(consultation_fee) AS minimum_fee,  
       MAX(consultation_fee) AS maximum_fee,  
       COUNT(*) AS total_doctors,  
       SUM(consultation_fee) AS total_fees,  
       AVG(consultation_fee) AS average_fee  
FROM doctors;
```

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
53 MAX(consultation_fee) AS maximum_fee,  
54 COUNT(*) AS total_doctors,  
55 SUM(consultation_fee) AS total_fees,  
56 AVG(consultation_fee) AS average_fee  
57 FROM doctors;
```

The Results window displays the following data:

minimum_fee	maximum_fee	total_doctors	total_fees	average_fee
600.00	1000.00	3	2400.00	800.000000

The Output window shows the execution log with the following entries:

#	Time	Action	Message	Duration / Fetch
33	09:43:55	SELECT specialization, COUNT(*) AS total_doctors FROM doctors GROUP BY spe...	3 row(s) returned	0.015 sec / 0.000 sec
34	09:44:18	SELECT specialization, COUNT(*) AS total_doctors FROM doctors GROUP BY spe...	3 row(s) returned	0.015 sec / 0.000 sec
35	09:44:39	MIN MAX COUNT SUM AVG	Error Code: 1064. You have an error in your SQL syntax; check the manual that corr...	0.031 sec
36	09:44:58	MIN MAX COUNT SUM AVG	Error Code: 1064. You have an error in your SQL syntax; check the manual that corr...	0.000 sec
37	09:45:01	MIN MAX COUNT SUM AVG	Error Code: 1064. You have an error in your SQL syntax; check the manual that corr...	0.000 sec
38	09:46:19	SELECT MIN(consultation_fee) AS minimum_fee, MAX(consultation_fee) AS m...	1 row(s) returned	0.000 sec / 0.000 sec

5. ORDER BY Clause

ORDER BY sorts the query result. Execute the following query in MySQL Workbench.

```
SELECT *  
FROM doctors  
ORDER BY consultation_fee DESC;
```

6. UNION Set Operation

UNION combines the results of two compatible SELECT queries and removes duplicate rows.

```
SELECT doctor_name FROM doctors WHERE specialization = 'Cardiology'  
UNION  
SELECT doctor_name FROM doctors WHERE specialization = 'Neurology';
```

7. INTERSECT Set Operation

INTERSECT returns rows common to both SELECT query results. If your MySQL version does not support INTERSECT, tell me and we can use the appropriate equivalent.

```
SELECT doctor_name FROM doctors WHERE consultation_fee >= 700  
INTERSECT  
SELECT doctor_name FROM doctors  
WHERE specialization IN ('Cardiology', 'Dermatology');
```

Note: Screenshots were provided for WHERE, GROUP BY, HAVING and Aggregate Functions. No screenshot for ORDER BY, UNION or INTERSECT was included in the uploaded set.

Conclusion

Thus, SQL filtering, grouping, conditional group filtering, aggregate functions, sorting, and set-operation queries were demonstrated for the appointment database.